Paper Trading Platform

Name(s): ``

Roll Number(s): ``

Public Git Repository: ``

> Submission note: this local folder is not currently a git repository (`git status` reports "not a git repository"), so official `git diff` files cannot be generated from this copy. Before final submission, push the project to a public GitHub/GitLab repository and generate per-file diffs from the repository history if this was modified from an original source.

1. Motivation and Project Overview

The goal of this project was to build a realistic paper trading platform where a user can practice stock trading without using real money. The application simulates the main workflows of a trading terminal: account creation, login, live watchlist prices, candlestick charts, order placement, order history, portfolio tracking, and historical portfolio reconstruction.

The motivation was to create something more practical than a static stock dashboard. A useful trading simulator needs both live market movement and persistent account state. For that reason, the project combines a FastAPI backend, a PostgreSQL database, a React frontend, WebSocket-based price updates, and a trading engine that records every order, trade, holding, and cash change.

The application is organized into two main parts:

- `backend/`: FastAPI application, PostgreSQL models, trading engine, market feed logic, authentication, watchlist APIs, and Alembic migrations.
- `frontend/`: React + TypeScript + Vite application that displays charts, watchlists, order tickets, portfolio pages, trade history, and authentication screens.

PostgreSQL is provided through `docker-compose.yml`, so the database can be started reproducibly without requiring a manual local installation.

2. System Architecture

The backend uses FastAPI as the HTTP and WebSocket server. The main application is created in `backend/app/main.py`, where routes are registered for REST APIs, authentication, trading operations, watchlist management, and streaming market data. The application also starts a market feed during its lifespan. Depending on configuration, the feed can be a random-walk simulator or Yahoo Finance-backed feed.

The central shared runtime state is defined in `backend/app/state.py`. It contains:

- a `CandleStore` for rolling OHLC candle data,
- a WebSocket hub for subscribed clients,
- a price feed instance,
- a database-backed `TradingEngine`.

The database layer is defined in `backend/app/db.py` and `backend/app/db_models.py`. SQLAlchemy is used for ORM models and session management. The helper `db_session()` wraps each database operation in a transaction, committing on success and rolling back on error. This keeps the code consistent and avoids connection leaks.

The frontend is a React TypeScript application. It uses Zustand stores for client-side state:

- `authStore.ts` persists login state and JWT tokens.
- `marketStore.ts` persists the selected symbol, selected chart interval, and current view.

The frontend communicates with the backend using `frontend/src/lib/api.ts` for HTTP requests and `frontend/src/lib/ws.ts` for WebSocket price ticks.

3. Functionality Implemented

3.1 Authentication

The project implements signup, login, and current-user APIs in `backend/app/api/auth.py`. Passwords are hashed using Passlib with bcrypt in `backend/app/auth/security.py`. JWT tokens are created on login/signup and validated through the dependency in `backend/app/auth/deps.py`.

On the frontend, `AuthPage.tsx` provides login and signup forms. After successful authentication, the token, user id, and email are stored through Zustand. Protected API requests attach the token as a Bearer token.

3.2 Market Data and Candlestick Charts

The project supports two feed modes:

- `MarketSimulator` in `backend/app/market/simulator.py`, which creates random-walk prices for default instruments.
- `YahooFeed` in `backend/app/market/yahoo_feed.py`, which polls Yahoo Finance for watchlist symbols and persists one-minute OHLC price history.

Each price tick updates the in-memory candle store. `backend/app/market/candles.py` builds candles for multiple intervals: `1m`, `5m`, `15m`, `1h`, and `1D`.

The frontend chart is implemented in `frontend/src/components/Chart.tsx` using `lightweight-charts`. It loads historical candles through `/api/candles/{symbol}` and then updates live using WebSocket tick messages. The chart includes local time formatting for Indian market usage.

3.3 Watchlist

The watchlist feature is implemented in `backend/app/api/watchlist.py` and `frontend/src/components/Watchlist.tsx`. Each user has a personal watchlist stored in the `watchlist` table. If a new user has no watchlist entries, the backend seeds default Indian stock symbols such as RELIANCE, TCS, INFY, HDFCBANK, ICICIBANK, SBIN, ITC, LT, WIPRO, and AXISBANK.

The user can search symbols using Yahoo Finance, add NSE/BSE stocks to the watchlist, remove symbols, and subscribe to live updates for watched symbols. The frontend debounces search input

to avoid unnecessary requests.

3.4 Trading Engine

The main business logic is in `backend/app/trading/engine.py`. The trading engine supports:

- market buy and sell orders,
- limit buy and sell orders,
- order rejection for invalid quantity, insufficient cash, or insufficient holdings,
- immediate execution when a limit order already crosses the current market price,
- open limit orders that fill later when a price tick crosses their limit,
- order cancellation,
- order modification for open limit orders,
- trade recording,
- holding and weighted-average price updates,
- current portfolio calculation,
- historical portfolio reconstruction.

Market orders fill immediately at the latest known price. Limit orders either fill immediately if the limit has already crossed the market, or remain `OPEN`. On every tick, `TradingEngine.on_tick()` checks open limit orders for that symbol and fills eligible orders.

The engine uses PostgreSQL row locks through `SELECT ... FOR UPDATE` when reading user rows and open orders. This prevents race conditions such as two simultaneous buy requests spending the same cash balance.

3.5 Orders, Trades, and Portfolio

The trading APIs are defined in `backend/app/api/trading.py`. They include:

- `POST /api/orders` to place an order,
- `GET /api/orders` to list orders,
- `DELETE /api/orders/{order_id}` to cancel an order,
- `PATCH /api/orders/{order_id}` to modify an open limit order,
- `GET /api/trades` to list trades with filters,
- `GET /api/portfolio` to view current cash and holdings,
- `GET /api/portfolio/at` to reconstruct the portfolio at a previous timestamp.

The current portfolio uses current prices from the active feed. Holdings show quantity, average cost, latest traded price, and unrealized P&L.;

The frontend includes several related components:

- `OrderTicket.tsx` for BUY/SELL, MARKET/LIMIT, quantity shortcuts, limit price step buttons, and cash/holding validation preview.

- `OrdersPanel.tsx` for order display, cancellation, and modification.
- `Portfolio.tsx` for a compact sidebar portfolio.
- `PortfolioPage.tsx` for a detailed holdings page with total equity, cash, invested value, and P&L.;
- `HistoryPage.tsx` for trades, all orders, rejected orders, and filters.

3.6 Historical Portfolio

One of the more advanced features is historical portfolio reconstruction. The backend stores `starting_cash` on the user record and records every executed trade in the `trades` table. To reconstruct a portfolio at time `T`, the backend:

1. Starts with `starting_cash`.
2. Loads all trades where `executed_at <= T`.
3. Replays buys and sells in chronological order.
4. Rebuilds cash and positions.
5. Looks up historical close prices from `price_history`.
6. Computes P&L; as of that timestamp.

The frontend component `HistoricalPortfolio.tsx` lets the user choose a custom date/time or presets such as one hour ago, one day ago, one week ago, or one month ago.

4. Database Design

The SQLAlchemy models define the following main tables:

- `users`: stores user id, email, password hash, cash, starting cash, and creation time.
- `instruments`: stores default tradable instruments.
- `orders`: stores every order with side, quantity, type, limit price, status, timestamp, and rejection reason.
- `trades`: stores executed fills.
- `holdings`: stores current quantity and average price per user and symbol.
- `price_history`: stores OHLC candles by symbol and timestamp.
- `watchlist`: stores each user's selected symbols.

Composite primary keys are used where the data naturally has one row per user and symbol, such as `holdings` and `watchlist`. The `orders` table has an index on `(symbol, status)` to make open-limit-order lookup efficient during price ticks. The `price_history` table uses `(symbol, ts)` as a lookup path for historical prices.

Alembic migration files are present under `backend/alembic/versions/`, including the initial schema and a migration for authentication fields.

5. Code Created and Important Files

Important backend files:

- `backend/app/main.py`: FastAPI app setup, CORS, route registration, feed startup/shutdown.

- `backend/app/config.py`: runtime settings such as market hours, tick interval, max candles, and feed source.

- `backend/app/db.py`: SQLAlchemy engine, session factory, transaction helper.

- `backend/app/db_models.py`: database models for users, instruments, orders, trades, holdings, price history, and watchlist.

- `backend/app/trading/engine.py`: core trading engine and portfolio reconstruction.

- `backend/app/api/trading.py`: trading REST APIs.

- `backend/app/api/auth.py`: signup, login, and current-user endpoints.

- `backend/app/api/watchlist.py`: watchlist CRUD and Yahoo symbol search.

- `backend/app/api/ws.py`: WebSocket subscription stream.

- `backend/app/market/yahoo_feed.py`: live Yahoo Finance polling and price persistence.

- `backend/app/market/simulator.py`: simulated market feed.

- `backend/app/market/candles.py`: OHLC candle generation.

Important frontend files:

- `frontend/src/App.tsx`: main layout and view switching.
- `frontend/src/components/AuthPage.tsx`: authentication UI.
- `frontend/src/components/Watchlist.tsx`: watchlist and symbol search.
- `frontend/src/components/Chart.tsx`: candlestick chart.
- `frontend/src/components/OrderTicket.tsx`: order entry form.
- `frontend/src/components/OrdersPanel.tsx`: order list, modify, and cancel controls.
- `frontend/src/components/Portfolio.tsx`: compact portfolio summary.
- `frontend/src/components/PortfolioPage.tsx`: detailed portfolio dashboard.
- `frontend/src/components/HistoryPage.tsx`: trade/order/rejection history.
- `frontend/src/components/HistoricalPortfolio.tsx`: historical portfolio viewer.
- `frontend/src/lib/api.ts`: typed HTTP client.
- `frontend/src/lib/ws.ts`: WebSocket client with reconnect and resubscribe behavior.
- `frontend/src/store/authStore.ts` and `frontend/src/store/marketStore.ts`: persisted client state.

6. Use of AI

AI was used as a support tool during the project rather than as the sole developer. The main design decisions, feature choices, debugging decisions, and final integration were done by the team. AI helped mainly in four practical ways: understanding errors, comparing possible implementation approaches, generating small code snippets that were then reviewed and adapted, and improving the wording/structure of documentation.

For example, when working on the backend trading engine, AI was used to discuss how market orders and limit orders should behave, especially for cases such as insufficient balance, insufficient holdings, and limit orders that should remain open until the price crosses. The final implementation was still adjusted manually to fit the existing database models, API routes, and frontend

expectations.

AI was also used while working on frontend pieces such as the order ticket, watchlist, and portfolio/history views. In these cases, AI helped suggest component structure, TypeScript types, and UI states, but the code had to be connected manually to the existing Zustand stores, API wrapper, and WebSocket client. Some generated ideas were simplified or changed to match the rest of the application.

In the final stage, AI was used to go through the existing codebase and help prepare this report. It summarized the project structure, identified the important files, and helped convert the implementation details into clear report language.

Prompts/instructions used with AI:

```text
Explain how to structure a FastAPI paper trading backend with users, orders, trades, holdings, and portfolio APIs.

How should a paper trading engine handle MARKET and LIMIT orders, including rejected orders and insufficient cash?

Help debug why my React TypeScript API client is not matching my Zustand auth store.

Suggest a clean React component structure for a trading dashboard with chart, watchlist, order ticket, orders panel, and portfolio view.

see my project structure, i need you to generate a report follow these instructions -

A project report summarizing what you did.

Report can be about 3 to 4 pages.

Report should clearly describe what all you did, including motivation, functionality implemented, code created, and how you tested your code.

If you used AI, clearly describe how you used AI. Document the prompts you used to generate code, or to explore existing code.

If you didn't use AI, explain why you didn't use it!

...

go threough my code base and generate a nice report
```

All AI-generated suggestions were treated as drafts. The team reviewed them, modified code where needed, connected the pieces to the existing project, and made the final decisions about what to keep.

7. Limitations and Future Work

The current project is functional, but there are several improvements that would make it stronger:

- Add automated backend tests using pytest.
- Add frontend component tests or end-to-end tests using Playwright.

- Clean `backend/requirements.txt`, because it appears to include many packages from the local Python environment that are not needed by this project.

- Add a root-level `.gitignore` to exclude `node_modules`, `__pycache__`, `.env`, and build output.

- Add a root-level README with exact setup steps.

- Ensure all Alembic migrations match the latest models, especially fields such as `starting_cash`, `watchlist`, and `price_history`.

- Push the project to a public GitHub/GitLab repository and include the public URL in this report.

- Generate required per-file diff files from git history if the project was modified from an original source.

8. Conclusion

This project implements a complete paper trading workflow with persistent users, live or simulated prices, charting, watchlists, market and limit orders, order history, current portfolio tracking, and historical portfolio reconstruction. The most important part of the system is the database-backed trading engine, which safely updates cash, holdings, orders, and trades inside transactions.

Overall, the project demonstrates full-stack development across database design, backend APIs, real-time WebSocket updates, frontend state management, and trading-specific business logic.